

Geração Combinatória

Subconjuntos e permutações em ordem lexicográfica

Prof. Dr. André Yoshiaki Kashiwabara

Algoritmos e Estruturas de Dados II — DCOMP/UFS

3 de setembro de 2026

1. Ordem lexicográfica e o tamanho do espaço de busca



2. Subconjuntos: vetor característico e máscara de bits



3. Subconjuntos por *backtracking* em ordem lex



4. Permutações por *backtracking* em ordem lex



5. Próxima permutação em $O(n)$ e análise de custo

Ordem lexicográfica e espaço de busca

Problema

Muitos problemas não têm algoritmo eficiente conhecido. A alternativa é *examinar todas as configurações candidatas*:

- subconjuntos: soma de subconjunto, mochila exata, cobertura;
- permutações: caixeiro viajante.

Solução

Um *gerador* que percorre o espaço de busca:

- sem repetir nenhuma configuração;
- sem esquecer nenhuma;
- em uma ordem *previsível* — a ordem lexicográfica.

Definição

Sejam $a = (a_1, \dots, a_p)$ e $b = (b_1, \dots, b_q)$ sequências sobre um conjunto ordenado. Dizemos que $a \prec_{\text{lex}} b$ quando:

- existe $i \leq \min(p, q)$ com $a_j = b_j$ para todo $j < i$ e $a_i < b_i$; **ou**
- $p < q$ e a é prefixo de b .

Intuição

É a ordem do dicionário: compara-se posição a posição e, na primeira diferença, decide-se. Quem acaba antes e é prefixo do outro vem primeiro.

Onde aparece: Knuth [?], seq. 7.2.1 (ordens de geração); Kreher & Stinson [?], cap. 2.

Comparações

- $(1, 3) \prec_{\text{lex}} (2)$ pois $1 < 2$
- $(1, 2) \prec_{\text{lex}} (1, 3)$ primeira diferença na 2ª posição
- $(1) \prec_{\text{lex}} (1, 2)$ prefixo vem antes
- $(2, 3) \prec_{\text{lex}} (3)$ pois $2 < 3$

Os 8 subconjuntos de $\{1, 2, 3\}$ em ordem lex

$$\emptyset \prec \{1\} \prec \{1, 2\} \prec \{1, 2, 3\} \prec \\ \{1, 3\} \prec \{2\} \prec \{2, 3\} \prec \{3\}$$

As 6 permutações de $\{1, 2, 3\}$ em ordem lex

$$123 \prec 132 \prec 213 \prec 231 \prec 312 \prec 321$$

n	2^n	$n!$
5	32	120
10	1 024	$3,6 \times 10^6$
15	32 768	$1,3 \times 10^{12}$
20	$1,0 \times 10^6$	$2,4 \times 10^{18}$

Consequência prática

Subconjuntos são viáveis até $n \approx 25$ –30; permutações, até $n \approx 11$ –13.

Qualquer trabalho *extra* por configuração multiplica um número já enorme: o gerador precisa ser econômico.

Onde aparece: Cormen et al. [?], Apêndice C.1 (contagem: permutações e combinações); Knuth [?], seq. 7.2.1.

O que aprendemos

- Enumeração exaustiva é a estratégia-base para problemas sem algoritmo eficiente.
- A ordem lexicográfica compara sequências posição a posição; prefixo vem antes.
- Gerar em ordem lex dá saída determinística, fácil de conferir e de comparar.
- 2^n subconjuntos e $n!$ permutações limitam severamente o n tratável.

Subconjuntos: vetor característico

Problema

Um subconjunto é um objeto de tamanho variável. Como percorrer *todos* eles sem estrutura de dados complicada?

Solução

Cada subconjunto vira um vetor de n bits — e portanto um inteiro entre 0 e $2^n - 1$. Enumerar subconjuntos vira *contar*.

Definição

Para $S \subseteq \{1, \dots, n\}$, o vetor característico é $c(S) = (c_1, \dots, c_n)$ com $c_i = 1$ se $i \in S$ e $c_i = 0$ caso contrário. A correspondência $S \leftrightarrow c(S)$ é uma bijeção entre os 2^n subconjuntos e os 2^n vetores binários de tamanho n .

c_1	c_2	c_3	c_4	c_5	
1	0	1	1	0	$\iff S = \{1, 3, 4\}$
1	2	3	4	5	$\iff 10110_2 = 22$

```
1  /* Percorre os 2^n subconjuntos de {1, ..., n}.
2     Bit mais significativo (posicao n-1) representa o elemento 1. */
3  void subconjuntos_mascara(int n)
4  {
5     for (unsigned m = 0; m < (1u << n); m++) {
6         printf("{");
7         for (int i = 1; i <= n; i++)
8             if (m & (1u << (n - i))) /* elemento i pertence a S? */
9                 printf(" %d", i);
10        printf(" }\n");
11    }
12 }
```

- Custo: $\Theta(n \cdot 2^n)$ — n testes de bit por subconjunto.
- Espaço adicional: $O(1)$. Limite prático: $n \leq 31$ com unsigned.

Onde aparece: Knuth [?], seq. 7.2.1.1 (geração de todas as n -uplas); Kreher & Stinson [?], seq. 2.2 (subsets).

Contagem binária ($m = 0, 1, \dots, 7$)

m	S
000	$\emptyset$
001	$\{3\}$
010	$\{2\}$
011	$\{2, 3\}$
100	$\{1\}$
101	$\{1, 3\}$
110	$\{1, 2\}$
111	$\{1, 2, 3\}$

Ordem lex dos *elementos*

$\emptyset, \{1\}, \{1, 2\}, \{1, 2, 3\},$
 $\{1, 3\}, \{2\}, \{2, 3\}, \{3\}$

Observação

A contagem binária é lex sobre os *vetores de bits*, não sobre as *listas crescentes de elementos*. Para obter a segunda ordem usamos *backtracking*.

O que aprendemos

- Subconjunto $\leftrightarrow$ vetor de n bits $\leftrightarrow$ inteiro em $[0, 2^n)$.
- Enumerar todos os subconjuntos é contar de 0 a $2^n - 1$: código curto, $O(1)$ de espaço.
- Custo $\Theta(n \cdot 2^n)$; cabe em `unsigned` até $n = 31$.
- A ordem produzida é *lex nos bits* — diferente da ordem *lex nos elementos*.

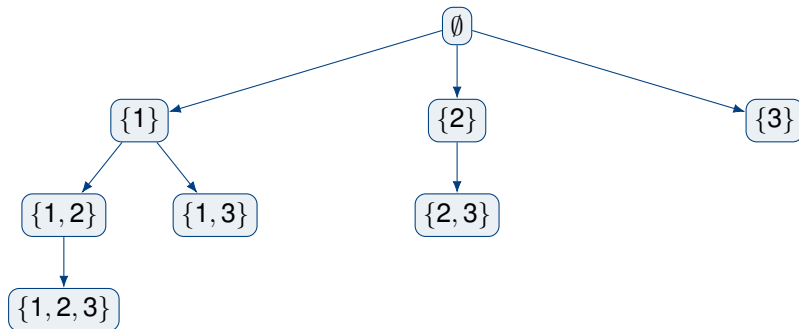
Subconjuntos por backtracking

Problema

Queremos a ordem lex sobre as listas crescentes de elementos — e queremos poder *podar* ramos inteiros quando o prefixo já é inviável (soma estourou, restrição violada). A máscara de bits não permite isso.

Solução

Construir o subconjunto incrementalmente: em cada nível, escolher o *próximo* elemento entre os maiores que o último escolhido, em ordem crescente. Visitar o nó ao chegar nele.



Percurso em pré-ordem, filhos da esquerda para a direita $\Rightarrow$ ordem lexicográfica.

```
1  int n;                /* universo {1, ..., n} */
2  int sol[MAXN];        /* prefixo em construcao */
3  int k;                /* tamanho do prefixo */
4
5  void subconjuntos(int inicio)
6  {
7      visita(sol, k);    /* 1. visita o no atual */
8      for (int i = inicio; i <= n; i++) { /* 2. filhos em ordem crescente */
9          sol[k++] = i;  /* escolhe i */
10         subconjuntos(i + 1); /* so elementos maiores que i */
11         k--;           /* desfaz a escolha */
12     }
13 }
14 /* chamada inicial: k = 0; subconjuntos(1); */
```

- $inicio = i + 1$ garante subconjunto *crescente* e sem repetição.
- Profundidade máxima da recursão: n ; espaço $O(n)$.

Onde aparece: Skiena [?], seq. 9.2.1; Kreher & Stinson [?], cap. 4; Ziviani [?], seq. 2.3.

Argumento

Em cada nó, o prefixo já fixado é comum a toda a subárvore. Os filhos são gerados com valores *i crescentes*, logo a subárvore do filho i contém apenas sequências menores (lex) que as da subárvore do filho $i + 1$. Como o nó é visitado *antes* dos filhos, o prefixo aparece antes de suas extensões — exatamente a segunda cláusula da definição de $\prec_{\text{lex}}$.

Exemplo: $n = 3$

Saída: $\emptyset$, $\{1\}$, $\{1, 2\}$, $\{1, 2, 3\}$, $\{1, 3\}$, $\{2\}$, $\{2, 3\}$, $\{3\}$.

Soma de subconjunto

Queremos os S com $\sum_{i \in S} w_i = A$, com todos os $w_i > 0$.

```
1 void busca(int inicio, int soma)
2 {
3     if (soma == A) visita(sol, k);
4     if (soma >= A) return;          /* poda: pesos positivos */
5     for (int i = inicio; i <= n; i++) {
6         sol[k++] = i;
7         busca(i + 1, soma + w[i]);
8         k--;
9     }
10 }
```

- Uma linha corta subárvores inteiras — impossível na enumeração por máscara.
- A ordem de visita dos ramos sobreviventes continua lexicográfica.

Onde aparece: Skiena [?], seq. 9.3 (*search pruning*); Kreher & Stinson [?], seq. 4.6; Cormen et al. [?], seq. 35.5.

O que aprendemos

- Escolher o próximo elemento sempre *maior* que o último evita repetições.
- Pré-ordem com filhos crescentes $\Rightarrow$ ordem lexicográfica dos elementos.
- Espaço $O(n)$; tempo $\Theta(n \cdot 2^n)$ no pior caso, muito menos com poda.
- A poda é a vantagem decisiva sobre a máscara de bits.

Permutações por backtracking

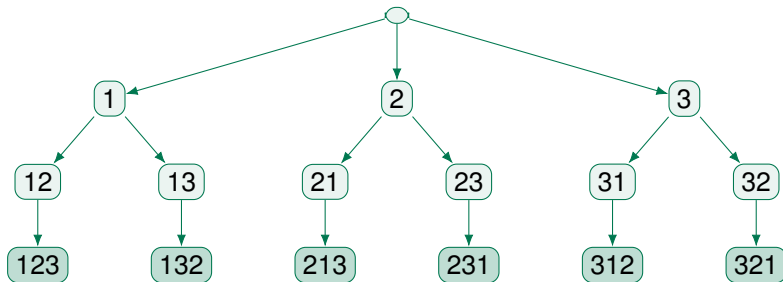
Problema

Em um subconjunto, cada elemento entra ou não. Em uma permutação, todos entram — o que muda é a *posição*. Precisamos saber quais valores ainda estão disponíveis.

Solução

Preencher as posições $0, 1, \dots, n - 1$ e, em cada posição, testar os valores livres em ordem crescente, marcando-os em um vetor `usado[]`.

A árvore de permutações de $\{1, 2, 3\}$



Só as folhas são visitadas: $n!$ folhas, todas na profundidade n .


```
1  int n;
2  int sol[MAXN];          /* permutacao parcial          */
3  int usado[MAXN + 1];    /* usado[v] = 1 se v ja esta em sol */
4
5  void permutacoes(int pos)
6  {
7      if (pos == n) { visita(sol, n); return; } /* folha */
8      for (int v = 1; v <= n; v++) {           /* valores em ordem crescente */
9          if (usado[v]) continue;
10         usado[v] = 1; sol[pos] = v;
11         permutacoes(pos + 1);
12         usado[v] = 0;                          /* desfaz */
13     }
14 }
15 /* chamada inicial: memset(usado, 0, sizeof usado); permutacoes(0); */
```

- Testar v em ordem crescente em cada posição $\Rightarrow$ folhas em ordem lex.
- Tempo $\Theta(n \cdot n!)$; espaço $O(n)$.

Onde aparece: Skiena [?], seç. 9.2.2 (*constructing all permutations*); Kreher & Stinson [?], seç. 2.4; Ziviani [?], seç. 2.3.

```
1 void permuta_troca(int v[], int pos, int n) /* NAO gera em ordem lex */
2 {
3     if (pos == n) { visita(v, n); return; }
4     for (int i = pos; i < n; i++) {
5         troca(&v[pos], &v[i]);
6         permuta_troca(v, pos + 1, n);
7         troca(&v[pos], &v[i]);
8     }
9 }
```

Saída para 123

123, 132, 213, 231, 321, 312

Dispensa usado [] e é mais rápida por constante, mas a troca *desordena* o sufixo. Use-a quando a ordem não importa.

Onde aparece: Sedgewick [?] (levantamento dos métodos por troca); Knuth [?], seq. 7.2.1.2, Algoritmo P (*plain changes*).

O que aprendemos

- Preencher posição a posição, testando valores livres em ordem crescente.
- `usado[]` custa $O(n)$ de espaço e mantém a ordem lexicográfica.
- Apenas folhas são visitadas: $n!$ saídas, custo $\Theta(n \cdot n!)$.
- A variante por trocas é mais enxuta, mas não respeita a ordem lex.

Próxima permutação e análise de custo

Problema

A recursão obriga a percorrer tudo de uma vez. Às vezes queremos *avançar uma permutação por vez*: dentro de um laço, retomando de onde parou, sem pilha de recursão.

Solução

Dado um arranjo, calcular *in loco* a menor permutação estritamente maior que ele — o sucessor lexicográfico, em $O(n)$. É o que faz `std::next_permutation`.

1	3	5	4	2
---	---	---	---	---

1. sufixo decrescente (5, 4, 2); pivô = 3 (último $v_i < v_{i+1}$)

1	3	5	4	2
---	---	---	---	---

2. mais à direita maior que o pivô: 4

1	4	5	3	2
---	---	---	---	---

3. troca pivô com ele: 1 4 5 3 2

1	4	2	3	5
---	---	---	---	---

4. inverte o sufixo: 1 4 2 3 5 é o sucessor

```
1 static void troca(int *a, int *b) { int t = *a; *a = *b; *b = t; }
2
3 static void inverte(int v[], int i, int j)
4 {
5     while (i < j) { troca(&v[i], &v[j]); i++; j--; }
6 }
7
8 int proxima_permutacao(int v[], int n)      /* 1 se avancou, 0 se era a ultima */
9 {
10     int i = n - 2;
11     while (i >= 0 && v[i] >= v[i + 1]) i--;      /* 1. pivo */
12     if (i < 0) return 0;                        /* sufixo e todo decrescente */
13     int j = n - 1;
14     while (v[j] <= v[i]) j--;                    /* 2. sucessor do pivo */
15     troca(&v[i], &v[j]);                        /* 3. troca */
16     inverte(v, i + 1, n - 1);                    /* 4. inverte o sufixo */
17     return 1;
18 }
```

Onde aparece: Knuth [?], seq. 7.2.1.2, Algoritmo L (*lexicographic permutation generation*); Kreher & Stinson [?], seq. 2.4 (sucessor lex). É a mesma rotina do `std::next_permutation` da biblioteca padrão do C++.

```
1 int v[] = {1, 2, 3, 4};  
2 do {  
3     visita(v, 4);  
4 } while (proxima_permutacao(v, 4));
```

Comece *ordenado* para obter todas as $n!$ permutações.

Por que é o sucessor

O sufixo à direita do pivô é decrescente: já é o maior arranjo daqueles valores, logo nada muda ali. Aumentar o pivô o mínimo possível e deixar o sufixo *crescente* dá a menor permutação acima da atual.

Método	Saídas	Tempo total	Espaço extra
Máscara de bits	2^n	$\Theta(n \cdot 2^n)$	$O(1)$
Subconjuntos (backtracking)	2^n	$\Theta(n \cdot 2^n)$	$O(n)$
Permutações (backtracking)	$n!$	$\Theta(n \cdot n!)$	$O(n)$
proxima_permutacao	$n!$	$\Theta(n!)$	$O(1)$

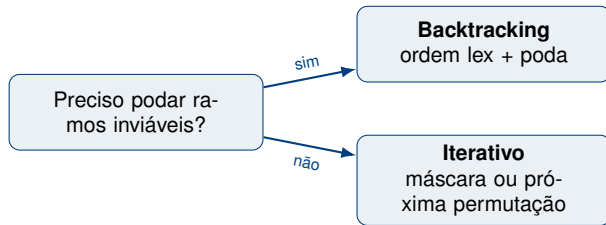
Custo amortizado do sucessor

Uma chamada custa $O(n)$ no pior caso, mas o pivô está na última posição em metade dos casos, na penúltima em $1/6$, e assim por diante. O custo médio é $\sum_{k \geq 1} k/k! = e \approx 2,7$ comparações: $O(1)$ **amortizado**.

Onde aparece: Knuth [?], seq. 7.2.1.2 (análise do Algoritmo L); Sedgewick [?] (comparação empírica dos geradores).

O que aprendemos

- O sucessor lex se calcula in loco em quatro passos: pivô, troca, inversão.
- Custo $O(n)$ por chamada e $O(1)$ amortizado; espaço extra $O(1)$.
- Permite laço iterativo, sem recursão e sem armazenar permutações.
- O gerador recursivo continua preferível quando há poda a fazer.



- Ordem lex é a mesma nos dois caminhos — muda o custo e a flexibilidade.
- Sempre estime 2^n ou $n!$ *antes* de rodar.

1. Escreva `proximo_subconjunto` que, dado um vetor crescente, produz o próximo subconjunto em ordem lex.
2. Adapte o gerador de subconjuntos para produzir só os de tamanho k , mantendo a ordem lex.
3. Resolva soma de subconjunto com poda e conte quantos nós a poda eliminou.
4. Meça o tempo de `proxima_permutacao` para $n = 8, 9, 10$ e confirme o fator n entre medições consecutivas.

Tutorial guiado: `tutorial/tutorial.ipynb`

Algoritmo da aula	Referência e local
Máscara de bits	Knuth [?], seq. 7.2.1.1; Kreher & Stinson [?], seq. 2.2
Subconjuntos por backtracking	Skiena [?], seq. 9.2.1; Ziviani [?], seq. 2.3
Subconjuntos de tamanho k	Kreher & Stinson [?], seq. 2.3; Knuth [?], seq. 7.2.1.3
Poda (soma de subconjunto)	Skiena [?], seq. 9.3; Kreher & Stinson [?], seq. 4.6; Cormen et al. [?], seq. 35.5
Permutações por backtracking	Skiena [?], seq. 9.2.2; Kreher & Stinson [?], seq. 2.4
Permutações por trocas	Sedgewick [?]; Knuth [?], seq. 7.2.1.2 (Algoritmo P)
<code>proxima_permutacao</code>	Knuth [?], seq. 7.2.1.2 (Algoritmo L); Kreher & Stinson [?], seq. 2.4
Contagem (2^n e $n!$)	Cormen et al. [?], Apêndice C.1

Onde aparece: O sucessor lexicográfico (Algoritmo L) é atribuído por Knuth a Narayana Pandita, na Índia do século XIV.

-  **Jon Bentley.**
Programming Pearls.
Addison-Wesley, 2 edition, 2000.
-  **Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan.**
Time bounds for selection.
Journal of Computer and System Sciences, 7(4):448–461, 1973.
Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.
-  **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.**
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Michael R. Garey and David S. Johnson.

Computers and Intractability: A Guide to the Theory of NP-Completeness.

W. H. Freeman, 1979.



David Gries.

The Science of Programming.

Springer-Verlag, 1981.



Michael Held and Richard M. Karp.

A dynamic programming approach to sequencing problems.

Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.



C. A. R. Hoare.

Algorithm 64: Quicksort.

Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.

Algorithm 65: FIND.

Communications of the ACM, 4(7):321–322, 1961.

O quickselect, publicado por Hoare no mesmo ano do quicksort.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Donald E. Knuth.

The Art of Computer Programming, Volume 3: Sorting and Searching.
Addison-Wesley, 2 edition, 1998.

Seção 5.3.3: seleção mínima em número de comparações.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.
Addison-Wesley, 1989.



David R. Musser.

Introspective sorting and selection algorithms.

Software: Practice and Experience, 27(8):983–993, 1997.

O introselect: híbrido entre quickselect e mediana das medianas.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.